

The Tech Stack, Explained Simply

No coding background required. This walks through every major piece of technology Fairmarket is built with, what job it does, why it was picked over the alternatives, and defines every confusing term (npm, API, database, and the rest) in plain English as it comes up. A full glossary is at the end for quick lookup.

Generated 2026-09-16

The Big Picture — Think of It Like a House

A website like Fairmarket is really three separate things working together, the same way a house has a foundation, the rooms you live in, and the street address that lets people find it:

- The Frontend — what a visitor actually sees and taps on: buttons, product photos, the search bar. This is the "rooms you live in."
- The Backend — where information is stored and kept safe: every product, every user account, every order. This is the "foundation."
- Hosting — the computer, run by someone else, that keeps the whole thing switched on and reachable 24/7 at a web address. This is the "street address."

Every technology named below fits into one of those three jobs. None of them were picked because they're trendy — each was picked because it does its one job better than the alternative, for a Nigerian marketplace specifically.

Frontend — What a Visitor Sees

Next.js (with React)

What it does: The toolkit that builds every page — the homepage, the product pages, the seller dashboard.

"React" is the part that draws buttons and forms; "Next.js" is the part that turns those into real, fast web pages and handles page navigation.

Why this one: It lets the same project also handle the server-side jobs below (see "One toolkit, two jobs"), instead of needing two entirely separate projects glued together.

Compared with: A plain React app with a separate backend service — rejected because it means building and maintaining two projects (login, security, data-fetching) instead of one.

TypeScript

What it does: A stricter version of JavaScript (the standard language web pages run on) that catches typos and mistakes — like accidentally treating a price as text instead of a number — before the site ever goes live, not after a customer hits the bug.

Why this one: For anything that touches money (checkout, commissions, refunds), catching mistakes before launch instead of after is worth the extra strictness.

Tailwind CSS

What it does: The system that controls how everything looks — colors, spacing, fonts — so all 54 pages share one consistent brand look without a designer manually styling each one.

Why this one: The whole indigo/cream Fairmarket color scheme is defined once, in one place, and every page automatically stays on-brand.

Backend — Where Information Lives

Supabase (built on Postgres)

What it does: The database — the actual filing cabinet holding every product, order, user account, and message. "Postgres" is the specific, extremely well-trusted filing-cabinet system used underneath; Supabase adds account sign-in and photo storage on top of it, ready-made.

Why this one: The security rule "a buyer can only see their own orders" is enforced by the filing cabinet itself, not just by the website's own code — so even a coding mistake elsewhere can't accidentally leak someone else's private information. This project's most important security decision, explained more in the Security section below.

Compared with: Firebase — a popular alternative — rejected because Fairmarket's data is deeply cross-linked (an order links to its items, its payment, its buyer, its seller, any dispute) in a way Postgres-style databases handle natively and Firebase-style ones don't.

Server Actions & API routes (part of Next.js)

What it does: The "staff behind the counter" — the code that actually processes a purchase, applies a coupon, or approves a seller, running privately on the server where a visitor can't tamper with it, rather than in the visitor's own browser.

Why this one: Anything involving money or permissions must run somewhere a customer can't open their browser's developer tools and rewrite the rules.

Hosting & Outside Services

Vercel

What it does: The company whose computers actually run the website around the clock and serve it to visitors at `fairmarketapp.vercel.app`. Also runs the one scheduled behind-the-scenes chore the site needs (automatically deleting old ID-verification documents after 90 days).

Why this one: It's built by the same company that builds Next.js, so deploying a Next.js site there needs no extra setup — and it's already set to run from a Frankfurt data center, close to where the database lives, so pages load quickly instead of a slow round trip to a different continent on every click.

Paystack

What it does: The payment processor — the company that actually moves money from a buyer's card or bank transfer into a seller's payout, safely.

Why this one: It's the most widely used Nigerian payment processor: Naira-native, with local bank-transfer and card support, and no foreign-currency conversion friction for either buyers or sellers.

Compared with: Stripe — a global alternative — rejected because its Nigerian bank-transfer and local-payment-method support is weaker; that matters more here than Stripe's broader reach into other countries.

GitHub

What it does: The permanent, timestamped history of every change ever made to the website's code — like a very detailed "track changes" for the entire project, plus a secure backup copy. This is also where the code physically lives so a developer (or Antigravity IDE) can open and edit it.

Why this one: It's the industry-standard place to store a project's code and history, and it's what Vercel watches to know when to publish a new version automatically.

A Note on Security

Three separate checks, stacked on top of each other, protect every piece of data — designed so a mistake in any one of them still leaves the others standing guard:

- The database itself refuses to hand over data that doesn't belong to whoever's asking — the deepest, most trustworthy layer.
- The website checks who's signed in before showing a page, so a stranger gets redirected to the login screen instead of an error.
- Every sensitive action (approving a seller, issuing a refund) separately double-checks the person doing it is actually allowed to.

On top of that: every login/signup/payment attempt is rate-limited (so an attacker can't just guess passwords thousands of times in a row), every admin action is permanently logged, and a user can request their data be deleted at any time — all already built and working.

Glossary — Plain-English Definitions

Terms that came up above, or that you'll likely hear from any future developer:

Term	In plain English
API	A menu of requests one piece of software can make to another — e.g. "give me this product's price" — without needing to know how the other side works internally.
npm	The tool that downloads and installs all the pre-built pieces of code ("packages") this project depends on, instead of writing everything completely from scratch.
Package / library	A chunk of ready-made code someone else wrote and shared, that this project reuses instead of rebuilding — e.g. the code that generates these PDFs.
Framework	A pre-built skeleton for an app (like Next.js) that already handles the common, repetitive parts, so developers only build what's actually unique to this project.
Repository ("repo")	The full folder of project code, plus its entire history of changes, stored on GitHub.
Deploy / deployment	The act of publishing a new version of the code so it's live on the real website, instead of just sitting on a developer's own computer.
Database	Organized, permanent storage for information — every user, product, and order lives in one.
Server	A computer, owned by Vercel in this case, that's always switched on and connected to the internet, waiting to respond to visitors.
Client / browser	The visitor's own device and web browser — the opposite end of the connection from the server.
Environment variable	A secret setting (like a payment API key) stored outside the code itself, so it never gets accidentally published or exposed publicly.
Webhook	An automatic notification one service sends another — e.g. Paystack silently telling Fairmarket "this payment just succeeded," without a human involved.
ORM (e.g. Prisma)	A tool that lets code talk to a database using everyday programming instead of raw database language. Prisma is a popular one — Fairmarket doesn't use it; it talks to Supabase's Postgres database directly, which is simpler for this project's needs.

RLS (Row Level Security)	The database's own built-in rule system deciding exactly who's allowed to see or change each individual piece of data — Fairmarket's core security layer, explained above.
JSON	A simple, universal text format for exchanging structured information between systems — e.g. how a completed order gets described when Paystack reports it.
SDK	A ready-made toolkit a company provides (e.g. Paystack's SDK) so a website can talk to that company's service correctly, without reverse-engineering it.